PYTHON FOR BEGINNERS · SESSION 10

Revision and Mini Project

Revise sessions 1 to 9, then build a student marks manager and a contact book step by step.

Recap

Ask the class:

- How do you count how many times each word appears?
- What is the difference between `d[key]` and `d.get(key)`?
- Which collection would you use for a phone book?

Today's plan

Today has **two parts**. First a quick **revision** of everything so far. Then a **mini project**, built one small step at a time, using only what we have learned.

1. **Revise** sessions 1 to 9 with short examples.
2. **Project 1:** a student marks manager.
3. **Project 2:** a contact book.
4. Practice and homework.

> **Remember**
> A program is built **in small steps**. Make each step work and print the result before you start the next one.

What we learned

SESSION	TOPIC	THE BIG IDEA
1	Introduction and setup	`print()`, running a `.py` file
2	Variables and data types	`int`, `float`, `str`, `bool`, `input()`, casting
3	Strings and operators	indexing, slicing, methods, f-strings, comparison and logical operators
4	Conditional statements	`if`, `elif`, `else`, nested conditions
5	Loops	`for`, `while`, `range()`, `break`, `continue`
6	Loop practice	nested loops, patterns, number programs
7	Lists	indexing, slicing, methods, loops over lists
8	Tuples and sets	immutability, unpacking, set operations
9	Dictionaries	key-value pairs, nested data, counting

Revision: variables and strings

```
name = "Aman"
age = 20
height = 5.9
student = True
print(type(name), type(age), type(height), type(student))
print(int("42") + 1, str(7) + "8", float("2.5"))
```

OUTPUT

```
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
43 78 2.5
```

```
word = "Python Programming"
print(word[0], word[-1], word[:6], word[::-1])
print(word.upper(), word.lower(), word.replace("P", "J"))
print(len(word), word.count("o"), word.split())
name = "Aman"
marks = 86.456
print(f"{name} scored {marks:.1f}")
```

OUTPUT

```
P g Python gnimmargorP nohtyP
PYTHON PROGRAMMING python programming Jython Jrogramming
18 2 ['Python', 'Programming']
Aman scored 86.5
```

Ask

What type is `input()` always? What do you do to do maths on it?

Revision: conditions and loops

```
marks = 72
if marks >= 90:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "F"
print(grade)
print(marks > 50 and marks < 80, not marks > 50)
```

OUTPUT

```
B
True False
```

```
for i in range(1, 6):
    if i == 4:
        continue
    print(i, end=" ")
print()

n = 5
total = 0
while n > 0:
    total += n
    n -= 1
print(total)
```

OUTPUT

```
1 2 3 5
15
```

```
for row in range(1, 5):
    print("*" * row)
```

OUTPUT

```
*
**
***
****
```

Revision: the four collections

	LIST	TUPLE	SET	DICTIONARY
Brackets	`[ ]`	`( )`	`{ }`	`{k: v}`
Find by	index	index	membership	**key**
Duplicates	allowed	allowed	**removed**	keys **unique**
Can change	**Yes**	**No**	**Yes**	**Yes**

```
nums = [4, 8, 2, 8, 6]
nums.append(10)
nums.sort()
print(nums, nums[-1], nums[1:3])

point = (3, 4)
x, y = point
print(x, y)

print(set(nums))
print({1, 2, 3} & {2, 3, 4})

marks = {"maths": 90, "science": 85}
marks["english"] = 78
print(marks.get("hindi", 0), sum(marks.values()))
```

OUTPUT

```
[2, 4, 6, 8, 8, 10] 10 [4, 6]
3 4
{2, 4, 6, 8, 10}
{2, 3}
0 253
```

Revision: find the output

Work out the output first, then press Run to check

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
t = (1, 2, 3)
x, *y = t
print(y)
s = {1, 2, 2, 3}
print(len(s))
d = {"a": 1}
d["b"] = d.get("b", 0) + 5
print(d)
```

Work out the output first, then press Run to check

```
text = "to be or not to be"
count = {}
for w in text.split():
    count[w] = count.get(w, 0) + 1
print(count)
print(max(count, key=count.get))
```

Project 1: student marks manager

We will build a small program that **stores the marks of students** and shows a **report**, the **topper** and the **ranking**.

What it must do

- **Add** a student with some marks, or add more marks later.
- Show a **report**: total, average and grade for each student.
- Show the **topper** and the **class average**.
- Show the **ranking** from best to worst.
- **Remove** a student.

Choosing the data

One **dictionary**: the **key** is the student name and the **value** is a **list of marks**.

```
students = {
    "Aman": [90, 80, 70],
    "Ankit": [85, 95, 88],
}
print(students["Ankit"])
print(students["Ankit"][0])
```

OUTPUT

```
[85, 95, 88]
85
```

Ask

Why a dictionary and not a list of lists? Think about how we find one student.

Step 1: adding students and marks

`setdefault(name, [])` creates an **empty list** for a new student and leaves an old one alone, so one line works for **both**.

```
students = {}
students.setdefault("Aman", []).extend([90, 80, 70])
students.setdefault("Ankit", []).extend([85, 95, 88])
students.setdefault("Aman", []).append(100)
print(students)
```

OUTPUT

```
{'Aman': [90, 80, 70, 100], 'Ankit': [85, 95, 88]}
```

Step 2: the report

Loop through `items()`, work out the **total**, the **average** and the **grade**, and print one neat row for each student.

```
students = {
    "Aman": [90, 80, 70],
    "Ankit": [85, 95, 88],
    "Kabir": [60, 55, 72],
}

print(f"{'Name':<8}{'Total':>6}{'Average':>9}  Grade")
for name, marks in students.items():
    total = sum(marks)
    avg = total / len(marks)
    if avg >= 85:
        grade = "A"
    elif avg >= 70:
        grade = "B"
    elif avg >= 50:
        grade = "C"
    else:
        grade = "F"
    print(f"{name:<8}{total:>6}{avg:>9.1f}  {grade}")
```

OUTPUT

```
Name     Total  Average  Grade
Aman       240     80.0  B
Ankit      268     89.3  A
Kabir      187     62.3  C
```

Note

`:<8` pads to 8 characters on the left, `:>6` on the right, and `:.1f` keeps one decimal.

Step 3: topper and ranking

First make a dictionary of **averages**, then use `max()` and `sorted()` with `key=averages.get`.

```
students = {
    "Aman": [90, 80, 70],
    "Ankit": [85, 95, 88],
    "Kabir": [60, 55, 72],
}

averages = {name: sum(m) / len(m) for name, m in students.items()}

topper = max(averages, key=averages.get)
print("Topper:", topper, round(averages[topper], 1))

class_avg = sum(averages.values()) / len(averages)
print("Class average:", round(class_avg, 1))

print("Ranking:")
ranked = sorted(averages, key=averages.get, reverse=True)
for rank, name in enumerate(ranked, start=1):
    print(rank, name, round(averages[name], 1))
```

OUTPUT

```
Topper: Ankit 89.3
Class average: 77.2
Ranking:
1 Ankit 89.3
2 Aman 80.0
3 Kabir 62.3
```

Ask

What does `reverse=True` do? What would the ranking be without it?

Step 4: commands in a loop

A real program keeps **asking** for commands inside a loop. Here the commands are in a **list** so you can run it in the browser. Next we replace the list with `input()`.

marks_manager.py (simulated commands)

```
students = {}
commands = [
    "add Aman 90 80 70",
    "add Ankit 85 95 88",
    "add Aman 100",
    "report",
    "topper",
    "remove Ankit",
    "remove Zed",
    "report",
    "dance",
    "quit",
]

for command in commands:
    print(">", command)
    parts = command.split()
    action = parts[0]

    if action == "add":
        name = parts[1]
        marks = [int(x) for x in parts[2:]]
        students.setdefault(name, []).extend(marks)
        print("Added marks for", name, marks)
    elif action == "report":
        for name, marks in students.items():
            print(name, sum(marks), round(sum(marks) / len(marks), 1))
    elif action == "topper":
        best = max(students, key=lambda n: sum(students[n]) / len(students[n]))
        print("Topper:", best)
    elif action == "remove":
        name = parts[1]
        if name in students:
            del students[name]
            print("Removed", name)
        else:
            print("No such student")
    elif action == "quit":
        print("Bye")
        break
    else:
        print("Unknown command")
```

OUTPUT

```
> add Aman 90 80 70
Added marks for Aman [90, 80, 70]
> add Ankit 85 95 88
Added marks for Ankit [85, 95, 88]
> add Aman 100
```

```
Added marks for Aman [100]
> report
Aman 340 85.0
Ankit 268 89.3
> topper
Topper: Ankit
> remove Ankit
Removed Ankit
> remove Zed
No such student
> report
Aman 340 85.0
> dance
Unknown command
> quit
Bye
```

- `split()` breaks a command into **words**; the first word is the **action**.
- `int(x)` turns each mark from text into a number.
- `break` ends the loop on `quit`; the last `else` catches **unknown** commands.

The full program with input()

This is the **complete marks manager**. Save it as `marks_manager.py` and run it on your computer, because it waits for you to **type**.

marks_manager.py (run on your computer)

```
students = {}

while True:
    print()
    print("1 Add student   2 Add marks   3 Report")
    print("4 Topper        5 Remove      6 Quit")
    choice = input("Choose: ")

    if choice == "1":
        name = input("Name: ")
        if name in students:
            print("Already there")
        else:
            students[name] = []
            print("Added", name)

    elif choice == "2":
        name = input("Name: ")
        if name not in students:
            print("No such student")
        else:
            mark = int(input("Mark: "))
            students[name].append(mark)

    elif choice == "3":
        if not students:
            print("No students yet")
        for name, marks in students.items():
            if marks:
                avg = sum(marks) / len(marks)
                print(f"{name:<8}{sum(marks):>6}{avg:>9.1f}")
            else:
                print(f"{name:<8}   no marks")

    elif choice == "4":
        with_marks = {n: sum(m) / len(m) for n, m in students.items() if m}
        if with_marks:
            best = max(with_marks, key=with_marks.get)
            print("Topper:", best, round(with_marks[best], 1))
        else:
            print("No marks yet")

    elif choice == "5":
        name = input("Name: ")
        if students.pop(name, None) is None:
            print("No such student")
        else:
            print("Removed", name)
```

```
    elif choice == "6":
        print("Bye")
        break

    else:
        print("Please type 1 to 6")
```

Careful

`int(input())` crashes if the user types a **word**. Next class we make the program safer with **functions**; for now, type numbers.

Project 2: contact book

A contact book stores **people and their details**. It uses a **nested dictionary**: the key is the name and the value is another dictionary with the **phone** and the **email**.

```
contacts = {
    "Aman": {"phone": "9876543210", "email": "aman@mail.com"},
    "Ankit": {"phone": "9123456780", "email": "riya@mail.com"},
}
print(contacts["Ankit"]["phone"])
for name, info in contacts.items():
    print(name, info["phone"], info["email"])
```

OUTPUT

```
9123456780
Aman 9876543210 aman@mail.com
Ankit 9123456780 riya@mail.com
```

What it must do

- **Add** a contact and **update** a detail.
- **Search** by name, even a part of the name.
- **Delete** a contact.
- **List** all contacts in alphabetical order.

Contact book: search, update and delete

Searching part of a name

```
contacts = {
    "Aman": {"phone": "9876543210", "email": "aman@mail.com"},
    "Amit": {"phone": "9000000001", "email": "amit@mail.com"},
    "Ankit": {"phone": "9123456780", "email": "riya@mail.com"},
}
text = "am"
found = [n for n in contacts if text.lower() in n.lower()]
print(found)
```

OUTPUT

```
['Aman', 'Amit']
```

Updating and deleting

```
contacts = {"Aman": {"phone": "9876543210", "email": "aman@mail.com"}}
contacts["Aman"]["phone"] = "9000000000"
contacts["Ankit"] = {"phone": "9123456780", "email": "riya@mail.com"}
del contacts["Ankit"]
print(contacts)
```

OUTPUT

```
{'Aman': {'phone': '9000000000', 'email': 'aman@mail.com'}}
```

Listing in order

```
contacts = {
    "Ankit": {"phone": "9123456780"},
    "Aman": {"phone": "9876543210"},
    "Kabir": {"phone": "9000000002"},
}
for name in sorted(contacts):
    print(name, contacts[name]["phone"])
```

OUTPUT

```
Aman 9876543210
Ankit 9123456780
Kabir 9000000002
```

Contact book: commands in a loop

contact_book.py (simulated commands)

```
contacts = {}
commands = [
    "add Aman 9876543210 aman@mail.com",
    "add Ankit 9123456780 riya@mail.com",
    "add Amit 9000000001 amit@mail.com",
    "search am",
    "update Aman phone 9000000000",
    "list",
    "delete Ankit",
    "delete Zed",
    "list",
    "quit",
]

for command in commands:
    print(">", command)
    parts = command.split()
    action = parts[0]

    if action == "add":
        contacts[parts[1]] = {"phone": parts[2], "email": parts[3]}
        print("Saved", parts[1])
    elif action == "search":
        found = [n for n in contacts if parts[1].lower() in n.lower()]
        print(sorted(found) if found else "Nothing found")
    elif action == "update":
        name, field, value = parts[1], parts[2], parts[3]
        if name in contacts and field in contacts[name]:
            contacts[name][field] = value
            print("Updated", name)
        else:
            print("Cannot update")
    elif action == "delete":
        if contacts.pop(parts[1], None) is None:
            print("No such contact")
        else:
            print("Deleted", parts[1])
    elif action == "list":
        for name in sorted(contacts):
            info = contacts[name]
            print(f"{name:<8}{info['phone']:<12}{info['email']}")
    elif action == "quit":
        print("Bye")
        break
```

OUTPUT

```
> add Aman 9876543210 aman@mail.com
Saved Aman
> add Ankit 9123456780 riya@mail.com
Saved Ankit
> add Amit 9000000001 amit@mail.com
Saved Amit
> search am
['Aman', 'Amit']
> update Aman phone 9000000000
Updated Aman
> list
Aman    9000000000  aman@mail.com
Amit    9000000001  amit@mail.com
Ankit   9123456780  riya@mail.com
> delete Ankit
Deleted Ankit
> delete Zed
No such contact
> list
Aman    9000000000  aman@mail.com
Amit    9000000001  amit@mail.com
> quit
Bye
```

Contact book: the full program

contact_book.py (run on your computer)

```
contacts = {}

while True:
    print()
    print("1 Add   2 Search   3 Update")
    print("4 Delete   5 List   6 Quit")
    choice = input("Choose: ")

    if choice == "1":
        name = input("Name: ")
        phone = input("Phone: ")
        email = input("Email: ")
        contacts[name] = {"phone": phone, "email": email}
        print("Saved", name)

    elif choice == "2":
        text = input("Search for: ").lower()
        found = [n for n in contacts if text in n.lower()]
        if found:
            for n in sorted(found):
                print(n, contacts[n]["phone"], contacts[n]["email"])
        else:
            print("Nothing found")

    elif choice == "3":
        name = input("Name: ")
        if name in contacts:
            field = input("phone or email: ")
            if field in contacts[name]:
                contacts[name][field] = input("New value: ")
                print("Updated")
            else:
                print("Type phone or email")
        else:
            print("No such contact")

    elif choice == "4":
        name = input("Name: ")
        if contacts.pop(name, None) is None:
            print("No such contact")
        else:
            print("Deleted", name)

    elif choice == "5":
        if not contacts:
```

```
            print("No contacts yet")
        for name in sorted(contacts):
            info = contacts[name]
            print(f"{name:<8}{info['phone']:<12}{info['email']}")

    elif choice == "6":
        print("Bye")
        break

    else:
        print("Please type 1 to 6")
```

Make it better

Real programs grow. Try these **extensions** on your own:

- **Marks manager:** give each student **several subjects** with a nested dictionary, and show the **highest mark** in each subject.
- **Marks manager:** list the students who **failed** (average below 50).
- **Contact book:** stop two contacts from having the **same phone number**.
- **Contact book:** search by **phone number** as well as by name.
- **Both:** save the data to a **file** so it is still there next time (we will learn files later).

> **Next class**
> Functions: the same code can be **packed into named blocks** so the menu stays short and tidy.

Practice

Revision questions

1. Take a number from the user and say whether it is **even or odd**.
2. Print the **multiplication table** of a number using a loop.
3. Find the **largest and smallest** number in a list without using `max()` and `min()`.
4. Remove the **duplicates** from a list and print it sorted.
5. Count the **vowels** in a sentence using a dictionary.

Project questions

6. In the marks manager, print the **grade** of every student next to their name.
7. Add a **"best"** command that prints the highest single mark and who got it.
8. In the contact book, add a **"count"** command that prints how many contacts there are.
9. In the contact book, make **search** work on the email as well.

Find the output

Work out the output first, then press Run to check

```
students = {"Aman": [90, 80], "Ankit": [70]}
students.setdefault("Kabir", []).append(60)
students["Ankit"].extend([80, 90])
print(len(students))
print(sum(students["Ankit"]))
print(sorted(students))
print("Zed" in students)
```

Try the best-mark question

Your turn: print the highest mark of each student, then press Run

```
students = {"Aman": [90, 80], "Ankit": [70, 85, 95], "Kabir": [60]}
# print each name with the highest mark
```

Homework and wrap-up

- Finish **one of the two projects** on your computer and run it with the `input()` menu.
- Add **one extension** from the "Make it better" list.
- Write a **small program of your own** that uses a dictionary, a list and a loop (for example a shopping list or a quiz score).
- Revise sessions **1 to 9** using the revision table, and write down **three things** you still find hard.

Next class

Functions.